Model Selection for Software Engineering and Agent Orchestration

A decision reference for routing software engineering tasks to the right model. Designed to be loaded by an orchestrator (Claude Code, custom agent, MCP-based router) so the agent can select models per task without human routing.

The doc is opinionated but multi provider by default. Different providers genuinely lead in different categories. Picking the strongest model per task beats defaulting to one family for everything.

Quick decision matrix

Task category	**Primary**	**Secondary**	**When to escalate**
Agent planning / decomposition	Gemini 3.5 Flash (Thinking)	Claude Sonnet 4.6	Use Opus 4.8 if the plan is irreversible
Multi file refactoring	Claude Opus 4.8	GPT 5.5	Run both in parallel for hardest refactors
Architectural design	Claude Opus 4.8	GPT 5.5	Always parallel for irreversible decisions
Daily implementation	Claude Sonnet 4.6	GPT 5.3 Codex	Opus only if Sonnet fails twice
Inline autocomplete	GitHub Copilot	Composer 2.5	Stay inline, don't escalate
Debugging (routine)	Claude Sonnet 4.6	GPT 5.5	Escalate to GPT 5.5 if Sonnet circles
Debugging (complex)	GPT 5.5	Claude Opus 4.8	Run both in parallel for production incidents
Autonomous engineering	Grok Build 0.1	GPT 5.5	Massive context, no output cap
Long context (whole repo)	Gemini 3.1 Pro	Claude Sonnet 4.6	Both support 1M+ context

Task category	Primary	Secondary	When to escalate
UI / frontend generation	v0 (Vercel)	Claude Sonnet 4.6	v0 for components, Sonnet for custom logic
Worker subagents (parallel)	Claude Haiku 4.5	DeepSeek V3	Escalate to Sonnet on retry
Cost-sensitive high volume	DeepSeek V3 / Coder	Claude Haiku 4.5	Sonnet only if quality matters
Tool use heavy	Claude Sonnet 4.6	GPT 5.5	Sonnet has best tool call reliability
Eval / judge	Cross provider	-	Always vary family from generator

1. Complex reasoning and system architecture

For high level planning, architectural design, and complex multi file debugging.

- **Primary: Claude Opus 4.8.** Gold standard for multi file refactoring and deep architectural planning. Strong at avoiding hallucinations when reviewing or writing large scale logic.
- **Secondary: GPT 5.5.** Leads broad CS reasoning and is exceptionally consistent across complex algorithms and instruction following.

When to use one vs both:

- One model: routine architecture work, tradeoff docs, normal refactor scope.
- Both in parallel: irreversible decisions, multi system migrations, performance critical refactors. Cost gap is real but cheap compared to a bad architectural decision.

2. Everyday coding and implementation

For inline completion, test writing, daily bug fixes. Speed and IDE integration matter more than heavy compute.

- **Primary: Claude Sonnet 4.6.** Developer favorite for daily implementation and tests. Best balance of reasoning speed and code quality.
- **Inline autocomplete: GitHub Copilot.** Industry standard, deeply integrated into VS Code, JetBrains, etc.

- **Inline in Cursor: Composer 2.5.** Free on most Cursor plans, tuned for the Cursor loop.
- **Alternative chat model: GPT 5.3 Codex.** OpenAI's code focused tier. Worth keeping in rotation when you want a different reasoning profile on a quick implementation.

3. Agentic and autonomous workflows

For agents that navigate the OS, run terminal commands, fix bugs without continuous hand holding. This is where model selection has the biggest leverage on cost and quality.

Top-level recommendations

- **Agent planner: Gemini 3.5 Flash (Thinking mode).** Strong decomposition reasoning at low latency and cost. Use as the orchestrator in any high volume agent system.
- **Autonomous engineering agent: Grok Build 0.1.** Built specifically for agentic engineering. Massive context windows, no output token limits, behaves well in long runs.
- **Tool calling reliability: Claude Sonnet 4.6.** Most reliable tool call generation across providers. Lowest rate of malformed calls. Use for any agent that exposes tools to a model.
- **Agentic IDE: Cursor.** Multi-file agent mode, MCP support, parallel agent sessions. Pair with whichever underlying model fits the task.

Agent orchestration roles

Role	**Recommended model**	**Why**
Planner	Gemini 3.5 Flash (Thinking)	Fast reasoning, cheap, strong on decomposition
Critic / replan check	Claude Sonnet 4.6	Different family from planner, catches blind spots
Worker (simple parallel)	Claude Haiku 4.5 or DeepSeek V3	Cheap, fast, fan out friendly
Worker (complex isolated)	Claude Sonnet 4.6	Strong tool use, good code generation
Worker (hardest leaf)	Claude Opus 4.8 or GPT 5.5	Reserved for genuinely hard subtasks
Tool execution	Claude Sonnet 4.6	Best track record on tool call reliability

Role	Recommended model	Why
Eval / judge	Cross provider mix	Sycophancy bias doesn't survive crossing providers

Multi-agent patterns

1. **Cross provider plan-execute-critique.** Gemini plans, Claude executes, GPT critiques. Each role uses the provider with the strongest profile.
2. **Hierarchical same family.** Opus at the top, Sonnet in the middle, Haiku at leaves. Consistent within family, escalating capability.
3. **Cheap then escalate.** Haiku or DeepSeek first pass, Sonnet on failure, Opus only if Sonnet also fails. Best for high volume production where most cases are easy.

4. UI and frontend generation

For converting visual designs to functional code or generating UI components.

- **Primary: v0 by Vercel.** Industry standard for component generation. Best output quality for React and Next.js stacks.
- **Secondary: Claude Sonnet 4.6.** For frontend code outside v0's component scope (state management, custom hooks, complex layouts).

5. Cost-sensitive and high-volume workloads

For high volume work where cost dominates quality concerns.

- **Primary: Claude Haiku 4.5.** Fastest option for lightweight lookups, simple refactoring, file structure checks. Strong instruction following despite the tier.
- **Cost extreme: DeepSeek V3 / Coder Series.** Cheapest credible tier. Use when processing millions of tokens per day and the task tolerates lower reasoning quality.

Debugging (cross category section)

Debugging spans multiple categories so worth pulling out specifically.

- **Routine bugs (off by one, simple logic, type errors): Claude Sonnet 4.6.**
- **Complex bugs (concurrency, distributed state, performance): GPT 5.5 as primary.** OpenAI's reasoning catches a class of bugs Claude tends to miss (state inconsistencies, race conditions, complex control flow). Run **GPT 5.5** and **Claude Opus 4.8** in parallel and merge findings for production incidents.
- **Long running incident analysis (huge log files, full traces): Gemini 3.1 Pro** for the long context ingestion, then **GPT 5.5** or **Opus 4.8** to reason about findings.

Cost-saving heuristics

1. **Start cheap, escalate on failure.** Haiku or DeepSeek first, Sonnet on retry, Opus only when Sonnet also fails.
2. **Use parallel work cheaply.** Subagents go to Haiku or DeepSeek unless reasoning is required.
3. **Cache prompts.** Prompt caching on Claude cuts costs by up to 90% on repeated context.
4. **Batch async work.** Batch API gives 50% discount on most providers.
5. **Mix providers across roles.** Cross provider routing reduces sycophancy bias and is often cheaper than running everything on one premium tier.
6. **Read with Flash, write with Sonnet, decide with Opus.** Tier the read/write/decide work appropriately.

Model identifiers

Anthropic Claude API:

```
None
claude-opus-4-8
claude-sonnet-4-6
claude-haiku-4-5
```

OpenAI:

```
None
gpt-5.5
gpt-5.3-codex
```

Google Gemini API:

```
None
gemini-3.5-flash
gemini-3.1-pro
```

xAI:

```
None
grok-build-0.1
```

DeepSeek:

```
None
deepseek-v3
deepseek-coder
```

Cursor (in-IDE model picker): select from chat or agent panel, or cycle with Cmd+/.

Orchestrator behavior notes

When this doc is loaded as a reference by an orchestrator:

1. **Default behavior**: use the Primary model for each task category.
2. **Fallback path**: if Primary fails twice (timeout, malformed output, off topic), escalate per the matrix.
3. **Parallel runs**: only invoke parallel cross-provider runs for tasks explicitly marked (architecture, complex debugging, eval).
4. **Cost guardrails**: route to Haiku or DeepSeek if estimated tokens exceed a per-task budget threshold, regardless of category.
5. **Tool use**: any task that requires tool calls routes to Claude Sonnet 4.6 unless the category specifies otherwise.

Where this guide will go stale

Update cycle: model releases happen monthly. Specific things to watch:

- New Claude versions (4.9, 5.0)
- GPT 5.6, 6.0
- Gemini 4.x
- Pricing shifts, especially around batch and caching discounts
- New benchmark leaders on SWE Bench, Terminal Bench, Aider

Check the providers' release notes before committing to a routing decision:

- https://platform.claude.com/docs/en/release-notes/overview
- https://platform.openai.com/docs/models
- https://ai.google.dev/gemini-api/docs/models

- https://cursor.com/docs/models-and-pricing